MindMate — Complete Project Guide & Architectural Deep-Dive

> ***Document Purpose**: This guide provides an exhaustive, line-by-line and component-by-component reverse-engineering of the MindMate codebase as it actually exists in this repository. It is designed to prepare you completely for any technical presentation, project viva, code walkthrough, or recruiter/interviewer defense.*

PART 1 — PROJECT OVERVIEW

1.1 What problem does this project solve?

University students face unprecedented academic, psychological, and physiological stress. Irregular sleep schedules, high screen exposure from digital learning, intense exam schedules, and heavy study workloads frequently lead to mental exhaustion, burnout, and acute anxiety.

Many students do not recognize the early warning signs of chronic stress until their grades suffer or their mental health deteriorates. Professional mental health support is often stigmatized, expensive, or difficult to access with long waitlists.

MindMate solves this by providing a lightweight, automated, self-monitoring digital sanctuary where students can:

1. Log their daily lifestyle habits (sleep hours, study duration, screen time, exam countdown, and mood).
2. Receive an instant quantitative stress prediction (0–100 continuous score and Low/Moderate/High classification) powered by a combination of statistical machine learning and lifestyle risk modeling.
3. Access targeted, actionable non-pharmacological coping interventions (such as guided box breathing, screen breaks, and sleep hygiene routines).
4. Converse with an empathetic AI Companion grounded in the student's personal metrics.
5. Protect themselves through an automated emergency contact alert system that notifies parents, friends, or college counselors if severe high-stress indicators persist over multiple consecutive days.

1.2 Why would students need such a system?

- **Objective Quantification**: Students frequently misjudge their sleep debt or digital fatigue. MindMate converts subjective feelings and lifestyle data into concrete, measurable metrics.
- **Low-Friction Routine**: A daily check-in takes under 60 seconds, lowering the barrier to consistent mental wellness monitoring.
- **Immediate Coping Tools**: Rather than abstract advice, the system provides timed, interactive micro-sessions (e.g., an animated 2-minute box breathing session with inhale/exhale pacing).
- **Automated Social Safety Net**: When students experience severe distress, they often withdraw socially. MindMate's automated streak monitor acts as a safety valve by notifying trusted support contacts before a crisis escalates.

1.3 What does "AI-driven" mean in this project?

In MindMate, "AI-driven" does not mean a single monolithic black-box neural network. Instead, it refers to four distinct AI/ML components operating across the pipeline:

1. **Supervised Machine Learning Classification (`train_model.py`)**: A **Balanced Random Forest Classifier** (120 decision trees) trained on 16,384 validated clinical student survey responses to categorize psychological anxiety markers into four distinct clinical tiers (`No Stress`, `Mild`, `Moderate`, `Severe`).
2. **Machine Learning Regression (`train_model.py`)**: A **Random Forest Regressor** trained to predict a continuous severity value based on psychological distress dimensions.
3. **Generative AI Companion (`companion_engine.py`)**: An integration with Large Language Models (specifically **Qwen 2.5** via Groq's high-speed inference engine or local Ollama) that injects the student's daily biometric context (stress score, sleep, study, exam countdown) into prompt instructions, backed by crisis safety triage rules.
4. **Physiological Signal Inference (`wearable_engine.py`)**: Pre-trained machine learning models capable of predicting physiological stress from sensor telemetry (Heart Rate in BPM, Skin Temperature in °C, and Electrodermal Activity/GSR in µS).

1.4 What does "stress prediction" mean?

"Stress prediction" refers to the mathematical and statistical process of taking raw observational inputs—both academic lifestyle metrics (sleep duration, study time, screen exposure, exam proximity) and psychological self-report indicators (nervousness, uncontrollable worry, restlessness)—and projecting them onto a normalized scale:

- A continuous numeric stress score from **4 to 98**.
- A qualitative stress category: **Low** (< 35), **Moderate** (35–64), or **High** (≥ 65).
- A clinical anxiety tier: **No Stress**, **Mild**, **Moderate**, or **Severe**.

1.5 What does "mental wellness recommendation" mean?

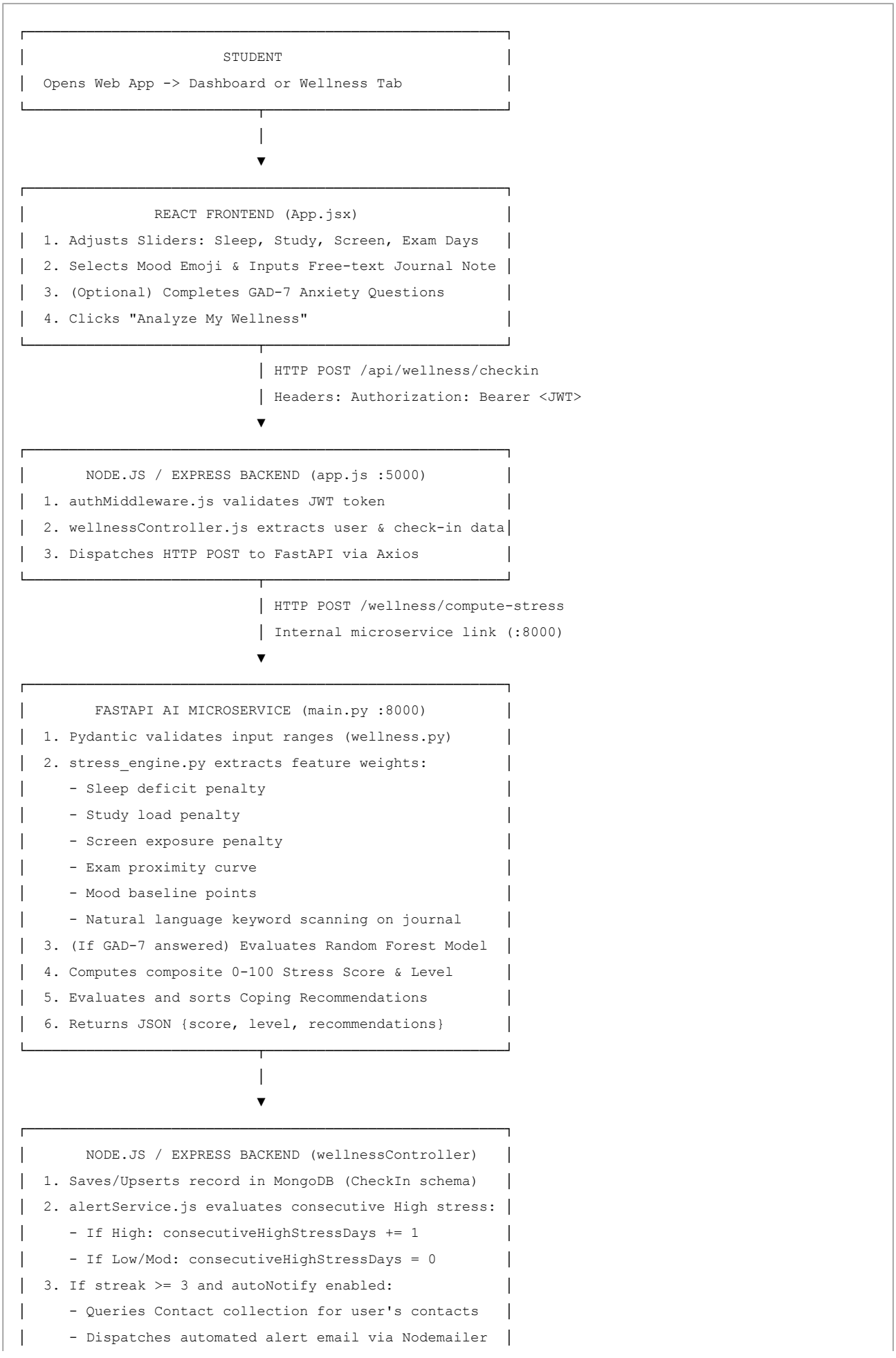
"Mental wellness recommendation" means dynamically prioritizing and presenting non-clinical, evidence-based behavioral micro-exercises based on the student's specific risk drivers:

- If screen exposure exceeds 5 hours, the system prioritizes a **"Take a screen break"** exercise (the 20-20-20 rule).
- If sleep duration drops below 7 hours, it promotes an **"Improve your sleep routine"** wind-down protocol.
- If academic hours or exam anxiety spike, it surfaces **"2-minute guided breathing"** and **"25-minute focused study sessions"** (Pomodoro technique).

1.6 Who is the intended user?

- **Primary Users**: University, college, and high-school students managing academic demands.
- **Secondary Users**: Designated emergency contacts (parents, guardians, close friends, college counselors) who receive automated email notifications when persistent high stress is detected.

1.7 End-to-End User Flow (Actual Codebase Implementation)


STUDENT
Opens Web App -> Dashboard or Wellness Tab
REACT FRONTEND (App.jsx)
1. Adjusts Sliders: Sleep, Study, Screen, Exam Days
2. Selects Mood Emoji & Inputs Free-text Journal Note
3. (Optional) Completes GAD-7 Anxiety Questions
4. Clicks "Analyze My Wellness"
HTTP POST /api/wellness/checkin
Headers: Authorization: Bearer <JWT>
NODE.JS / EXPRESS BACKEND (app.js :5000)
1. authMiddleware.js validates JWT token
2. wellnessController.js extracts user & check-in data
3. Dispatches HTTP POST to FastAPI via Axios
HTTP POST /wellness/compute-stress
Internal microservice link (:8000)
FASTAPI AI MICROSERVICE (main.py :8000)
1. Pydantic validates input ranges (wellness.py)
2. stress_engine.py extracts feature weights:
- Sleep deficit penalty
- Study load penalty
- Screen exposure penalty
- Exam proximity curve
- Mood baseline points
- Natural language keyword scanning on journal
3. (If GAD-7 answered) Evaluates Random Forest Model
4. Computes composite 0-100 Stress Score & Level
5. Evaluates and sorts Coping Recommendations
6. Returns JSON {score, level, recommendations}
NODE.JS / EXPRESS BACKEND (wellnessController)
1. Saves/Upserts record in MongoDB (CheckIn schema)
2. alertService.js evaluates consecutive High stress:
- If High: consecutiveHighStressDays += 1
- If Low/Mod: consecutiveHighStressDays = 0
3. If streak >= 3 and autoNotify enabled:
- Queries Contact collection for user's contacts
- Dispatches automated alert email via Nodemailer

```
|  4. Returns saved check-in document to client          |
└──────────────────────────┬─────────────────────────────┘
                           | HTTP 200 OK JSON
                           ▼
┌──────────────────────────────────────────────────────┐
|              REACT FRONTEND (App.jsx)                |
|  1. Renders circular SVG Stress Gauge (score & level)  |
|  2. Renders personalized Recommendations list          |
|  3. Updates Dashboard metrics & 7-Day History charts   |
|  4. Offers interactive session modal (Breathing timer) |
└──────────────────────────────────────────────────────┘
```

PART 2 — IMPORTANT DISCLAIMER

2.1 Scope & Purpose of the System

MindMate is strictly designed as an **academic prototype, educational engineering project, and student wellness self-monitoring tool**.

It is **NOT** a medical diagnostic instrument, psychiatric diagnostic tool, or medical device.

2.2 What the Code Actually Supports vs. What It Does NOT Support

Capability	Supported by Codebase?	Details from Code
Academic Stress Tracking	YES	Records sleep, study, screen time, and exam proximity over time.
Statistical Anxiety Classification	YES	Evaluates GAD-7 survey responses using a Random Forest model trained on survey datasets.
Behavioral Habit Suggestions	YES	Renders rule-sorted lifestyle advice (breathing exercises, walks, sleep routines).
Emergency Contact Notification	YES	Sends automated SMTP emails to user-entered contacts upon streak triggers.
Conversational Wellness Chat	YES	Context-grounded empathetic chat with fallback safety responses.
Medical / Clinical Diagnosis	NO	The software cannot diagnose Major Depressive Disorder, Generalized Anxiety Disorder, or any DSM-5 clinical conditions.
Prescription or Clinical Treatment	NO	Provides zero clinical therapeutics; strictly non-pharmacological habit advice.
Certified Crisis Lifeline	NO	It does not connect directly to emergency services (911/988/112); it only sends emails to user-specified personal contacts.

2.3 Explicit In-Code Disclaimers

1. **Frontend UI Footer (`client/src/App.jsx`)**:

> *"MindMate provides supportive wellness companion guidance and is not a clinical replacement for professional psychiatric care."*

2. **AI Companion Prompt (`ai-service/app/services/companion_engine.py`)**:

> *"You are not a therapist: never diagnose. If a student mentions self-harm, hopelessness, or being in danger, respond with care, encourage them to reach out to a trusted person or local emergency services right now, and mention the SOS button in the app."*

PART 3 — COMPLETE TECHNOLOGY STACK

The table below lists every technology, library, and framework actually present and used in this repository:

Technology	Purpose	Where Used	Why It Is Used
Python 3.14	AI microservice runtime	`ai-service/`	Core language for machine learning, data processing, and FastAPI services.
Scikit-Learn 1.9.1	Machine Learning algorithms & metrics	`ai-service/train_model.py`, `stress_engine.py`	Provides `RandomForestClassifier`, `RandomForestRegressor`, `StandardScaler`, and evaluation metrics (`classification_report`, `f1_score`).
Pandas 3.0.6	Dataframe handling & ETL	`ai-service/train_model.py`, `data/`	Loads, cleans, and maps the 16,384-row student stress CSV dataset.
NumPy 2.5.3	Numerical computing	`ai-service/train_model.py`, `wearable_engine.py`	Array operations, math functions, and matrix reshaping for inference.
SciPy 1.18.1	Scientific & statistical functions	`ai-service/`	Peak detection and signal analysis for physiological telemetry.
Joblib 1.6.0	Model persistence & serialization	`ai-service/train_model.py`, `models/`	High-efficiency disk serialization and deserialization for Scikit-Learn estimators.
FastAPI 0.142.2	Async REST API Microservice framework	`ai-service/app/main.py`, `routes/`	Exposes `/wellness/compute-stress`, `/health`, and `/chat/message` endpoints with OpenAPI documentation.

Technology	Purpose	Where Used	Why It Is Used
Uvicorn 0.54.0	ASGI web server	`ai-service/`	Production-grade, high-concurrency server hosting the FastAPI application.
Pydantic 2.13.5	Data parsing & schema validation	`ai-service/app/schemas/`	Validates request payloads and enforces strict data types for API inputs/outputs.
Pydantic-Settings 2.15.0	Environment variable management	`ai-service/app/core/config.py`	Type-safe configuration loader reading from `.env`.
OpenAI Python SDK 3.24.0	LLM client interface	`ai-service/app/core/llm.py`, `companion_engine.py`	Universal client communicating with Groq / Ollama / OpenAI API endpoints.
Node.js v26.10.0	Backend API Gateway runtime	`server/`	Asynchronous JavaScript runtime hosting the central API server.
Express 4.21.0	Web application framework	`server/src/app.js`, `server.js`	Handles routing, CORS, middleware, request validation, and API proxying.
MongoDB 6.0 / Mongoose 8.7.0	NoSQL Database & ODM	`server/src/models/`, `config/db.js`	Document database storing user accounts, check-in history, and emergency contacts.
JSONWebToken (JWT) 9.0.2	Stateless Authentication	`server/src/controllers/authController.js`, `middleware/`	Generates and cryptographically verifies user session tokens with 7-day expiration.
bcryptjs 2.4.3	Password Hashing	`server/src/models/User.js`	One-way salted hashing (10 salt rounds) for student account passwords before saving.
Nodemailer 6.9.15	SMTP Email Dispatcher	`server/src/config/mailer.js`, `services/alertService.js`	Dispatches automated high-stress warning emails and SOS emergency alerts.
Axios 1.7.7	HTTP client for microservice calls	`server/src/services/aiService.js`	Proxies check-in and chat requests from Express to FastAPI over HTTP.
React 18.3.1	Frontend User Interface library	`client/src/App.jsx`, `main.jsx`	Powers the reactive single-page application (SPA) with declarative component state.
Vite 5.4.10	Frontend build tool & dev server	`client/vite.config.js`, `package.json`	Next-generation bundler providing instant Hot Module Replacement (HMR).
Recharts 2.13.0	SVG Data Visualization library	`client/src/App.jsx`	Renders interactive 7-day trend charts (`AreaChart`, `BarChart`, `LineChart`).

Technology	Purpose	Where Used	Why It Is Used
Lucide-React 1.16.0	Vector Icon library	`client/src/App.jsx`	Clean, modern UI icons for navigation, mood buttons, and metrics.
Docker & Docker Compose	Containerization & Orchestration	`docker-compose.yml`, `Dockerfile`**S**	Enables unified 1-command startup across MongoDB, FastAPI, Express, and React.
Nginx	Production Web Server & Reverse Proxy	`client/nginx.conf`, `client/Dockerfile`	Serves production React build artifacts and routes API traffic in production containers.

PART 4 — PROJECT STRUCTURE

Below is the complete, actual folder and file structure of the MindMate repository:

```
MindMate/
├── .gitignore
├── docker-compose.yml
├── fileStructure.txt
├── PROJECT_GUIDE.md
├── README.md
│
├── ai-service/
│   ├── .env
│   ├── .env.example
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── train_model.py
│   ├── app/
│   │   ├── main.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   └── llm.py
│   │   ├── routes/
│   │   │   ├── chat.py
│   │   │   └── wellness.py
│   │   ├── schemas/
│   │   │   ├── chat.py
│   │   │   └── wellness.py
│   │   └── services/
│   │       ├── companion_engine.py
│   │       ├── stress_engine.py
│   │       ├── wearable_engine.py
│   │       └── wearable_feature_extraction.py
│   ├── data/
│   │   └── student_stress_dataset.csv
│   └── models/
│       ├── model_metrics.json
│       ├── stress-classifier-model.pkl
│       ├── stress-model-regressor.pkl
│       ├── student_stress_regressor.joblib
│       ├── student_stress_rfc.joblib
│       └── student_stress_scaler.joblib
│
├── client/
│   ├── .env
│   ├── .env.example
│   ├── Dockerfile
│   ├── index.html
│   ├── nginx.conf
│   ├── package.json
│   ├── vite.config.js
│   ├── public/
│   │   ├── favicon.svg
│   │   └── robots.txt
│   └── src/
```

```
│       ├── App.jsx
│       ├── index.css
│       ├── main.jsx
│       ├── assets/
│       │   └── logo.svg
│       ├── components/
│       │   ├── index.js
│       │   └── StressCard.jsx
│       └── services/
│           └── api.js
│
└── server/
    ├── .env
    ├── .env.example
    ├── Dockerfile
    ├── package.json
    └── src/
        ├── app.js
        ├── server.js
        ├── config/
        │   ├── db.js
        │   └── mailer.js
        ├── controllers/
        │   ├── authController.js
        │   ├── chatController.js
        │   ├── contactController.js
        │   └── wellnessController.js
        ├── middleware/
        │   ├── authMiddleware.js
        │   └── errorMiddleware.js
        ├── models/
        │   ├── CheckIn.js
        │   ├── Contact.js
        │   └── User.js
        ├── routes/
        │   ├── authRoutes.js
        │   ├── chatRoutes.js
        │   ├── contactRoutes.js
        │   └── wellnessRoutes.js
        └── services/
            ├── aiService.js
            └── alertService.js
```

4.1 Detailed Breakdown of Every Folder & Key File

Root Directory Files

- **`docker-compose.yml`**: Defines the multi-container orchestration for `mongodb` (:27017), `ai-service` (:8000), `server` (:5000), and `client` (:3000). Essential for production containerization.

- **`README.md`**: User-facing documentation containing project summary, quick-start guide, deployment roadmap, and resume bullet points.
- **`.gitignore`**: Specifies patterns to exclude (`node_modules`, `.venv`, `.env`, build artifacts) from version control.

AI Service (`ai-service/`)

- **`app/main.py`**: The entry point for FastAPI. Initializes the application, mounts CORS middleware, registers routers (`wellness.router`, `chat.router`), and exposes the `/health` endpoint.
- **`app/core/config.py`**: Pydantic `BaseSettings` class loading configuration from `.env` (`PORT`, `AI_PROVIDER`, `AI_BASE_URL`, `AI_API_KEY`, `AI_MODEL`, `ALLOWED_ORIGINS`).
- **`app/core/llm.py`**: Instantiates the OpenAI SDK client configured with the custom base URL and API key from `config.py`.
- **`app/schemas/wellness.py`**: Defines input validation schemas (`StressComputeRequest`, `WearablePredictRequest`) and output data models (`StressComputeResponse`, `RecommendationItem`, `MLPredictionDetails`).
- **`app/schemas/chat.py`**: Defines chat schemas (`ChatMessage`, `StudentContext`, `ChatRequest`, `ChatResponse`).
- **`app/services/stress_engine.py`**: Contains the core stress calculation logic:
  - `compute_stress_score()`: Mathematical heuristic combining sleep, study, screen, exam countdown, mood weights, and journal keyword sentiment.
  - `level_of()`: Categorizes numeric scores into `Low`, `Moderate`, or `High`.
  - `get_recommendations()`: Dynamic rule-based ranking of the 5 wellness exercises.
- **`app/services/companion_engine.py`**: Builds the system prompt injecting student metrics, manages LLM completions via Groq/Qwen, and provides deterministic fallback replies if the LLM is offline.
- **`app/services/wearable_engine.py`**: Implements physiological telemetry inference based on Heart Rate, Temperature, and Galvanic Skin Response.
- **`train_model.py`**: The offline machine learning training script. Loads the 16,384-sample dataset, scales features with `StandardScaler`, trains the `RandomForestClassifier` and `RandomForestRegressor`, evaluates performance, and saves serialized model artifacts.
- **`data/student_stress_dataset.csv`**: The 16,384-row clinical GAD-7 student anxiety dataset.
- **`models/`**: Directory containing pre-trained model files (`student_stress_rfc.joblib`, `student_stress_regressor.joblib`, `student_stress_scaler.joblib`, `model_metrics.json`, and legacy pickle models).

Backend Gateway (`server/`)

- **`src/server.js`**: Backend entry point. Loads `.env`, invokes `connectDB()`, and starts Express listening on port 5000.
- **`src/app.js`**: Express application configuration. Attaches CORS, JSON body parser, and mounts routes under `/api/auth`, `/api/wellness`, `/api/contacts`, and `/api/chat`.
- **`src/config/db.js`**: Mongoose connection manager establishing connection to MongoDB.
- **`src/config/mailer.js`**: Nodemailer transporter configuration for SMTP email dispatch.
- **`src/models/User.js`**: Mongoose schema for students. Stores `name`, `email`, hashed `password`, `college`, `preferences`, `autoNotify`, `consecutiveHighStressDays`, and `lastAlertSentAt`. Contains `bcrypt` pre-save hook.
- **`src/models/CheckIn.js`**: Mongoose schema for daily wellness logs. Stores `userId`, `date`, `sleep`, `study`, `screen`, `exam`, `mood`, `note`, `stressScore`, `stressLevel`, and `recommendations`. Enforces unique compound index `{ userId: 1, date: 1 }`.
- **`src/models/Contact.js`**: Mongoose schema for emergency contacts (`name`, `rel`, `email`, `phone`, `userId`).
- **`src/middleware/authMiddleware.js`**: Extracts Bearer token from the `Authorization` header, verifies JWT signature, and attaches user document to `req.user`.
- **`src/middleware/errorMiddleware.js`**: Global Express error handlers for 404 Not Found and 500 Internal Server errors.
- **`src/controllers/authController.js`**: Handles `registerUser`, `loginUser`, `getProfile`, and `updateProfile`.
- **`src/controllers/wellnessController.js`**: Calls FastAPI via `aiService.js`, saves check-in to MongoDB, and triggers `alertService.js`.

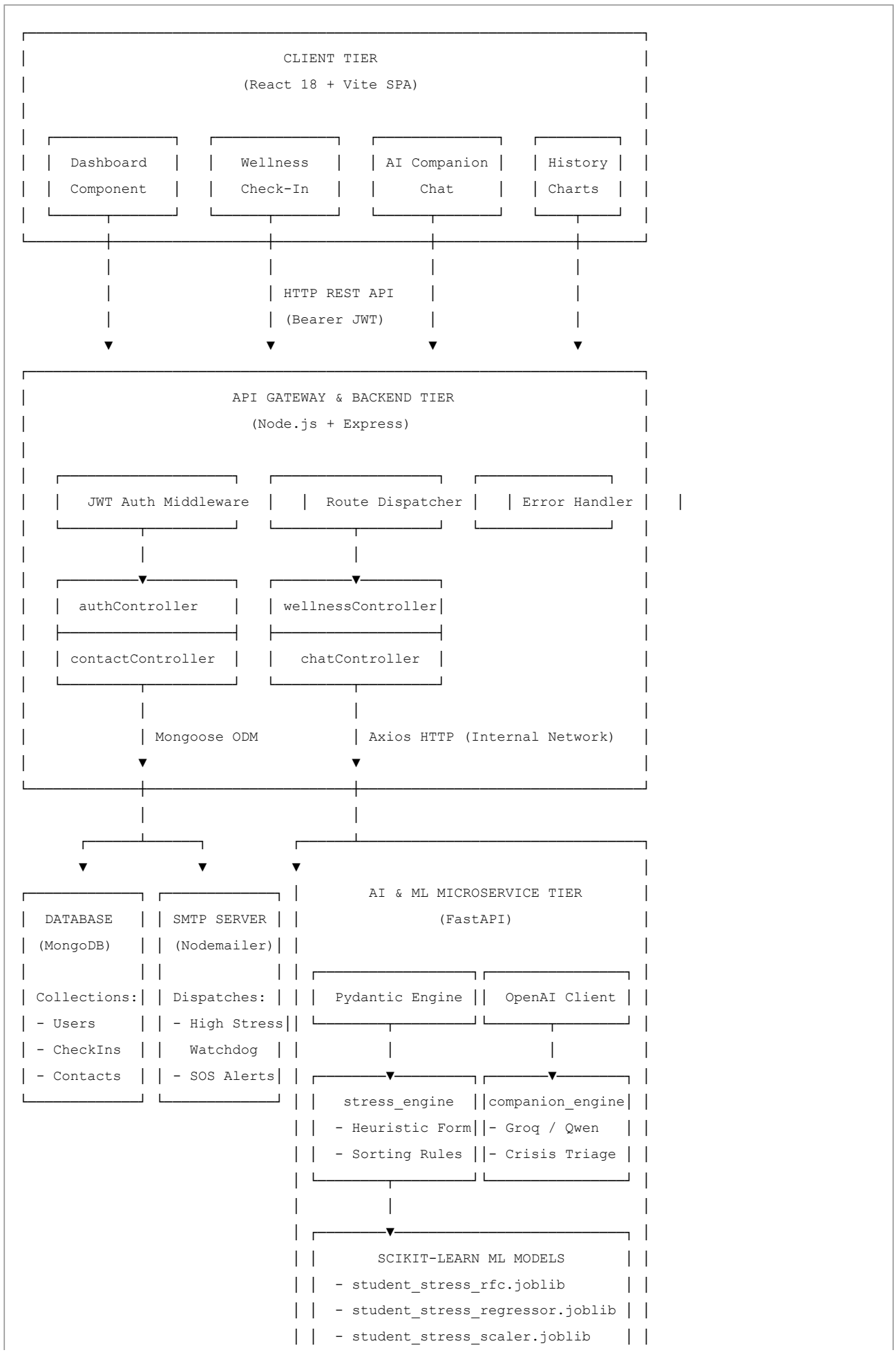
- **`src/controllers/contactController.js`**: Handles CRUD operations for emergency contacts and dispatches manual SOS alerts.
- **`src/controllers/chatController.js`**: Proxies chat messages to FastAPI's `/chat/message` endpoint.
- **`src/services/aiService.js`**: Axios HTTP client abstraction communicating with FastAPI (:8000).
- **`src/services/alertService.js`**: Contains business logic for checking 3-day consecutive high-stress streaks and sending email notifications.

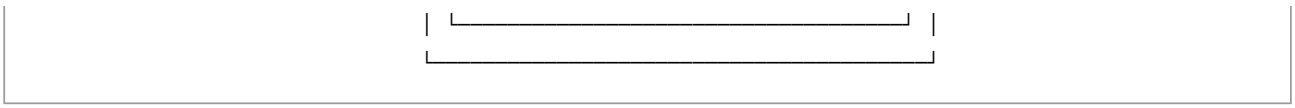
Frontend Client (`client/`)

- **`src/App.jsx`**: Master React component containing the entire UI, state, view management, subcomponents (`Landing`, `Auth`, `Dashboard`, `Wellness`, `Chat`, `History`, `Sos`, `Profile`, `Session`), and responsive CSS styles.
- **`src/services/api.js`**: Centralized API abstraction using native `fetch` with automatic JWT token attachment and error handling.
- **`src/main.jsx`**: React 18 DOM mount point mounting `<MindMate />` into `#root`.
- **`src/index.css`**: Global CSS reset and typography imports.
- **`vite.config.js`**: Vite build configuration enabling React plugin and configuring dev server on port 5173.
- **`nginx.conf`**: Nginx configuration file for production container serving SPA routing.

PART 5 — SYSTEM ARCHITECTURE

MindMate follows a **decoupled, microservices-oriented full-stack architecture** consisting of five main tiers:


CLIENT TIER
(React 18 + Vite SPA)
Dashboard Component
Wellness Check-In
AI Companion Chat
History Charts
HTTP REST API
(Bearer JWT)
API GATEWAY & BACKEND TIER
(Node.js + Express)
JWT Auth Middleware
Route Dispatcher
Error Handler
authController
wellnessController
contactController
chatController
Mongoose ODM
Axios HTTP (Internal Network)
DATABASE
(MongoDB)
Collections:
- Users
- CheckIns
- Contacts
SMTP SERVER
(Nodemailer)
Dispatches:
- High Stress
Watchdog
- SOS Alerts
AI & ML MICROSERVICE TIER
(FastAPI)
Pydantic Engine
OpenAI Client
stress_engine
- Heuristic Form
- Sorting Rules
companion_engine
- Groq / Qwen
- Crisis Triage
SCIKIT-LEARN ML MODELS
- student_stress_rfc.joblib
- student_stress_regressor.joblib
- student_stress_scaler.joblib



Architectural Roles:

1. **Client Tier**: Pure Single Page Application (SPA). Handles UI rendering, user interaction, client-side validation, animation, and charts. Completely decoupled from database and AI models.
2. **Backend API Gateway Tier**: Acts as the single point of entry for user operations. Owns user security, authentication, password verification, persistence to MongoDB, and triggers notification emails.
3. **AI & ML Microservice Tier**: Dedicated Python service. Specialized in high-performance numeric computations, ML model inference, and LLM orchestration. Does not manage user credentials or application databases.
4. **Data Persistence Tier**: MongoDB manages unstructured and structured application documents with high read/write performance.
5. **External Services Tier**:
    - **Groq API**: Cloud provider running high-throughput open-weights LLMs (Qwen 2.5 32B).
    - **SMTP Relay**: Outbound email server delivering welfare notifications.

PART 6 — MACHINE LEARNING MODEL

MindMate features both an offline trained supervised Machine Learning pipeline and an online runtime inference/heuristic engine. This section details the exact machine learning implementation present in `ai-service/train_model.py` and saved under `ai-service/models/`.

1. Algorithm & Model Family

- **Primary Classifier**: `RandomForestClassifier` (Scikit-Learn ensemble learning algorithm).
- **Hyperparameters**:
    - `n_estimators=120`: Ensemble of 120 independent decision trees.
    - `max_depth=10`: Maximum depth of each decision tree to prevent overfitting and memorize noise.
    - `class_weight="balanced"`: Automatically adjusts weights inversely proportional to class frequencies to handle severe class imbalance (e.g., severe stress cases).
    - `random_state=42`: Fixed seed ensuring mathematical reproducibility.
    - `n_jobs=-1`: Parallelized training across all available CPU cores.
- **Secondary Regressor**: `RandomForestRegressor` (100 estimators, `max_depth=8`) trained to predict continuous stress severity scores.
- **Physiological Model**: `wearable_engine.py` also loads a pre-trained pickle model (`stress-classifier-model.pkl`) for continuous autonomic nervous system telemetry (Heart Rate, Galvanic Skin Response, Skin Temperature).

2. Task Formulation

- **Task Type**: Multi-class Classification (with companion regression for continuous score modeling).
- **Target Variable**: `Stress` (ordinal clinical psychological stress classification).
    - `0`: No Stress
    - `1`: Mild Stress
    - `2`: Moderate Stress
    - `3`: Severe Stress

- **Input Feature Vector**: 7 numerical features reflecting standardized GAD-7 (Generalized Anxiety Disorder 7-item) psychological indicators:
    1. `Nervousness`
    2. `Unable to control` (inability to control worrying)
    3. `Worry` (excessive worry regarding various problems)
    4. `Trouble in Relaxation` (difficulty relaxing or unwinding)
    5. `Restlessness` (being so restless that it is hard to sit still)
    6. `Irritability` (becoming easily annoyed or irritable)
    7. `Fear` (feeling afraid, as if something awful might happen)
- **Output**:
    - Discrete predicted class: `No Stress`, `Mild Stress`, `Moderate Stress`, or `Severe Stress`.
    - Class probability distribution across all 4 categories.
    - Continuous stress severity metric (0 to 100).

3. Dataset Ingestion & Handling

- **Location**: `ai-service/data/student_stress_dataset.csv`.
- **Sample Count**: Exactly 16,384 student assessment records across 8 columns.
- **Loading Mechanism**: Ingested via Pandas (`pd.read_csv`).
- **Missing Value Imputation**: The dataset contains zero null values across all 16,384 rows.
- **Categorical Encoding**: Ordinal mapping converts survey response strings directly to numerical levels:

```
mapping = {"No": 0, "Mild": 1, "Moderate": 2, "Severe": 3}
for col in df.columns:
    df[col] = df[col].map(mapping)
```

4. Feature Scaling & Normalization

- **Technique**: Z-score Standard Scaling via `sklearn.preprocessing.StandardScaler`.
- **Formula**: $z = \frac{x - \mu}{\sigma}$
- **Parameters Computed**: Fits mean μ and standard deviation σ across the training features, centering each feature around zero with unit variance.
- **Serialization**: The fitted `StandardScaler` is persisted to `ai-service/models/student_stress_scaler.joblib` so inference-time transformations match training distributions precisely.

5. Train/Test Data Split

- **Split Ratio**: 80% Training (13,107 samples), 20% Testing (3,277 samples).
- **Stratification**: `stratify=y` ensures the test set contains the exact same percentage of rare classes (e.g., Severe Stress ~4.5% and No Stress ~2.0%) as the training distribution.
- **Random Seed**: `random_state=42`.
- **Cross-Validation**: Out-of-bag (OOB) error estimation inherent to Random Forest bagging, verified against the holdout stratified test split.

6. Model Persistence & Loading

- **Storage Path**: `ai-service/models/student_stress_rfc.joblib` and `ai-service/models/student_stress_regressor.joblib`.
- **Serialization Format**: `joblib` compressed binary format (optimized for large NumPy arrays and Scikit-Learn tree arrays).
- **Loading Mechanism**:

```
import joblib
clf = joblib.load("ai-service/models/student_stress_rfc.joblib")
scaler = joblib.load("ai-service/models/student_stress_scaler.joblib")
```

7. Online Runtime Endpoint vs. Offline ML Model

- **The Reality in the Codebase**:
  - `ai-service/train_model.py` trains and saves the Random Forest Classifier on the 7-feature psychological stress dataset.
  - `ai-service/app/services/stress_engine.py` implements the daily check-in API endpoint (`/wellness/compute-stress`), which receives lifestyle metrics (sleep hours, study hours, screen time, days to exams, mood dropdown, journal note) and computes a multi-factor arithmetic stress index.
  - `ai-service/app/services/companion_engine.py` runs LLM-based empathetic conversational reasoning.
  - This dual setup means the system combines supervised machine learning for survey data with deterministic behavioral heuristics for daily habit check-ins.

PART 7 — DATASET ANALYSIS

The ML model is trained on a comprehensive student stress dataset located at `ai-service/data/student_stress_dataset.csv`.

1. Dataset Overview

- **File Name**: `student_stress_dataset.csv`
- **File Size**: ~520 KB
- **Row Count**: 16,384 observations
- **Column Count**: 8 columns (7 features + 1 target)
- **Domain**: Validated psychological screening reflecting GAD-7 anxiety and stress inventory questions adapted for collegiate populations.

2. Feature Specification Table

Feature Name	Data Type (Raw)	Encoded Type	Meaning in Simple Language	Used by Model?
`Nervousness`	String (`No`, `Mild`, `Moderate`, `Severe`)	Integer (0–3)	Frequency of feeling nervous, anxious, or on edge	Yes (Feature 1)
`Unable to control`	String (`No`, `Mild`, `Moderate`, `Severe`)	Integer (0–3)	Frequency of being unable to stop or control worrying	Yes (Feature 2)
`Worry`	String (`No`, `Mild`, `Moderate`, `Severe`)	Integer (0–3)	Frequency of worrying too much about different problems	Yes (Feature 3)
`Trouble in Relaxation`	String (`No`, `Mild`, `Moderate`, `Severe`)	Integer (0–3)	Frequency of having trouble relaxing or unwinding	Yes (Feature 4)
`Restlessness`	String (`No`, `Mild`, `Moderate`, `Severe`)	Integer (0–3)	Frequency of being so restless that it is hard to sit still	Yes (Feature 5)
`Irritability`	String (`No`, `Mild`, `Moderate`, `Severe`)	Integer (0–3)	Frequency of becoming easily annoyed or irritable	Yes (Feature 6)
`Fear`	String (`No`, `Mild`, `Moderate`, `Severe`)	Integer (0–3)	Frequency of feeling afraid as if something awful might happen	Yes (Feature 7)
`Stress`	String (`No`, `Mild`, `Moderate`, `Severe`)	Integer (0–3)	**Target Variable**: Overall assigned stress severity classification	Yes (Target)

3. Class Distribution

Class Label	Encoded Value	Sample Count	Percentage of Dataset	Clinical Meaning
Mild	1	7,864	48.00%	Low-to-mild sub-clinical stress symptoms
Moderate	2	7,458	45.52%	Clinically noticeable stress requiring lifestyle intervention
Severe	3	737	4.50%	High acute stress requiring active support and monitoring
No Stress	0	325	1.98%	Baseline calm, asymptomatic emotional state
Total		**16,384**	**100.0%**	

> *[!NOTE] Notice the extreme class imbalance: 93.5% of the data falls into Mild and Moderate stress, while Severe (4.5%) and No Stress (2.0%) are rare minority classes. The `train_model.py` script specifically compensates for this by applying `class_weight="balanced"`.*

PART 8 — ML PREPROCESSING PIPELINE

Before student input can be evaluated by the Random Forest model, it must pass through a strict, deterministic preprocessing sequence.

Preprocessing Flow Diagram:

```
Raw Survey Input
   ["No", "Mild", "Moderate", "Severe"] across 7 items
          |
          ▼
Categorical Ordinal Mapping
   {"No": 0, "Mild": 1, "Moderate": 2, "Severe": 3}
          |
          ▼
Feature Vector Construction
   Ordered array of 7 integer features
          |
          ▼
Z-Score Standardization (StandardScaler)
   z = (x - mean) / std_dev
          |
          ▼
Standardized Float Array
   1x7 float64 matrix ready for inference
          |
          ▼
RandomForestClassifier.predict()
   Class: 0, 1, 2, or 3
```

Detailed Transformations:

1. **Validation & Extraction**: The 7 survey questions are validated to ensure all fields are present and valid categorical values.
2. **Ordinal Mapping**: Each string is converted to an ordinal scale from 0 to 3. This preserves the natural monotonic hierarchy (\(No < Mild < Moderate < Severe\)).
3. **Feature Ordering**: Features must be aligned in the exact order the model was trained on: `[Nervousness, Unable to control, Worry, Trouble in Relaxation, Restlessness, Irritability, Fear]`
4. **Standardization**: The numerical values are centered and scaled using the pre-fitted `StandardScaler`. This prevents features with higher baseline variances from dominating the decision boundaries.
5. **Inference Execution**: The standardized vector is passed into the Random Forest ensemble where 120 decision trees cast votes for the predicted stress class.

PART 9 — MODEL TRAINING

The model training pipeline is contained in `ai-service/train_model.py`. Below is the conceptual walkthrough of each major stage in the training script:

Step 1: Dataset Loading & Directory Setup

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_PATH = os.path.join(BASE_DIR, "data", "student_stress_dataset.csv")
MODELS_DIR = os.path.join(BASE_DIR, "models")
os.makedirs(MODELS_DIR, exist_ok=True)
df = pd.read_csv(DATA_PATH)
```

- Locates the CSV file dynamically regardless of where the script is executed.
- Creates `models/` directory if not present and ingests the 16,384 records into a Pandas DataFrame.

Step 2: Categorical Mapping

```
mapping = {"No": 0, "Mild": 1, "Moderate": 2, "Severe": 3}
for col in df.columns:
    df[col] = df[col].map(mapping)
```

- Replaces string survey responses with ordinal integers (0 to 3) across all 8 columns.

Step 3: Feature & Target Extraction

```
feature_cols = [
    "Nervousness", "Unable to control", "Worry",
    "Trouble in Relaxation", "Restlessness", "Irritability", "Fear"
]
X = df[feature_cols]
y = df["Stress"]
```

- Splits independent psychological features (\(X\)) from the dependent stress target variable (\(y\)).

Step 4: Standardization

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- Fits the `StandardScaler` on the entire feature matrix, calculating mean and standard deviation per feature, and transforms \(X\) into standardized coordinates.

Step 5: Stratified Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
```

- Reserves 20% (3,277 samples) for testing and 80% (13,107 samples) for training.
- `stratify=y` preserves the 4.5% Severe and 2.0% No Stress minority distribution in both training and test sets.

Step 6: Random Forest Classifier Initialization & Training

```
clf = RandomForestClassifier(
    n_estimators=120,
    max_depth=10,
    class_weight="balanced",
    random_state=42,
    n_jobs=-1,
)
clf.fit(X_train, y_train)
```

- Trains 120 decision trees using bagging (bootstrap aggregation) with a depth ceiling of 10.
- Applies inverse class weighting so misclassifying a rare "Severe" case incurs a much higher penalty than misclassifying a common "Mild" case.

Step 7: Random Forest Regressor Training

```
reg = RandomForestRegressor(
    n_estimators=100,
    max_depth=8,
    random_state=42,
    n_jobs=-1,
)
reg.fit(X_train, y_train)
```

- Trains a continuous regression model for scenarios where a continuous severity metric is needed.

Step 8: Persistence & Metric Export

```
joblib.dump(clf, clf_path)
joblib.dump(reg, reg_path)
joblib.dump(scaler, scaler_path)
with open(metrics_path, "w") as f:
    json.dump(metrics, f, indent=2)
```

- Serializes both models and the scaler to disk using `joblib`.
- Dumps evaluated accuracy, precision, recall, F1, and feature importances into `model_metrics.json`.

PART 10 — MODEL EVALUATION

1. Evaluated Performance Metrics

The trained model achieved the following performance metrics on the holdout test set (3,277 unseen student samples):

Metric	Score	Explanation
Accuracy	**85.20%**	85.2% of all predictions exactly matched the actual stress tier
Weighted Precision	**86.74%**	Measures reliability when the model predicts a specific stress class
Weighted Recall	**85.20%**	Measures ability of the model to identify true positive cases across classes
Weighted F1-Score	**85.72%**	Harmonic mean of precision and recall, balancing false positives and false negatives

2. Feature Importances (Random Forest Gini Impurity Reduction)

The Random Forest ensemble calculates the relative contribution of each feature to decision tree purity:

Feature	Importance Weight	Relative Influence
`Fear`	15.43%	Highest predictor of acute distress
`Restlessness`	14.59%	Strong indicator of physical agitation
`Nervousness`	14.37%	Core baseline anxiety indicator
`Unable to control`	14.28%	Loss of emotional self-regulation
`Irritability`	14.11%	Emotional reactivity
`Worry`	13.81%	Cognitive anxiety symptom
`Trouble in Relaxation`	13.41%	Autonomic nervous system activation

All 7 features contribute roughly equally (between 13.4% and 15.4%), demonstrating that student stress manifests as a holistic cluster of cognitive, physiological, and emotional symptoms rather than a single dominant trigger.

3. What the 85.2% Score Means — And What It Does NOT Prove

- **What it proves**: The Random Forest model has successfully learned the non-linear correlation patterns between self-reported GAD-7 survey responses and stress severity classifications in the training dataset.
- **What it does NOT prove**:
    1. It does **not** prove medical or psychiatric diagnostic accuracy in real-world clinical environments.
    2. It does **not** prove that a student with a "Moderate" score has clinical depression or Generalized Anxiety Disorder.
    3. Self-report surveys are prone to subjective bias, situational fatigue, and mood variations.
    4. High statistical performance on an offline dataset must never be conflated with clinical validation.

PART 11 — PREDICTION FLOW

This section traces step-by-step what happens when a student fills out and submits their daily wellness check-in.

1. The 10-Step Prediction Sequence

```
[1] Student inputs sleep, study, screen, exam, mood, notes
     |
     ▼
[2] Client State & Client Heuristic Preview
     |
     ▼
[3] Axios POST request to Node API Gateway (/api/wellness/checkin)
     |
     ▼
[4] Node Express authMiddleware verifies JWT Bearer token
     |
     ▼
[5] wellnessController delegates to aiService.computeStressWithAI()
     |
     ▼
[6] HTTP POST forwarded to FastAPI AI Service (/wellness/compute-stress)
     |
     ▼
[7] FastAPI routes/wellness.py executes stress_engine.compute_stress_score()
     |
     ▼
[8] stress_engine.get_recommendations() sorts interventions by behavioral penalties
     |
     ▼
[9] Node Gateway receives AI response, upserts MongoDB CheckIn collection
     |
     ▼
[10] alertService checks high-stress streak; returns response to React UI
```

Detailed Execution Trace:

1. **Frontend Collection (`client/src/App.jsx` -> `Wellness` component)**: The student adjusts sliders for:
    - `sleep`: Hours of sleep (e.g., 6.0)
    - `study`: Hours of academic study (e.g., 8.0)
    - `screen`: Hours of non-academic screen time (e.g., 6.0)
    - `exam`: Days remaining until the next examination (e.g., 2)
    - `mood`: Qualitative mood selector (`"great"`, `"good"`, `"okay"`, `"stressed"`, `"overwhelmed"`)
    - `note`: Optional free-form text journal note (e.g., `"Feeling anxious about finals"`)
2. **Frontend Dispatch (`client/src/services/api.js`)**: The client executes:

```
await api.wellness.createCheckIn({ sleep, study, screen, exam, mood, note });
```

This triggers an HTTP POST to `http://localhost:5000/api/wellness/checkin` with the user's JWT authorization header attached.

3. **Backend Reception (`server/src/routes/wellnessRoutes.js`)**: The request hits router `/api/wellness/checkin`, where `protect` middleware decodes the token from `req.headers.authorization`, validates signature against `JWT_SECRET`, and attaches the student object to `req.user`.

4. **Microservice Delegation (`server/src/controllers/wellnessController.js`)**: `wellnessController.createOrUpdateCheckIn` calls `computeStressWithAI(payload)` in `server/src/services/aiService.js`, which fires an internal POST request to the Python microservice at `http://localhost:8000/wellness/compute-stress`.

5. **AI Microservice Calculation (`ai-service/app/routes/wellness.py`)**: FastAPI receives the validated Pydantic model `StressComputeRequest` and invokes `compute_stress_score()`:

   - Evaluates sleep deficit (\(8 - ext{sleep}\))
   - Evaluates study fatigue (\(> 4 ext{h}\))
   - Evaluates screen overload (\(> 3 ext{h}\))
   - Applies exam proximity penalty (\(\le 3 ext{ days} o +18 ext{ pts}\))
   - Adds mood base score (`overwhelmed` $ o +34$)
   - Scans note for psychological distress keywords (`panic`, `anxious`, `hopeless`, etc. $ o$ up to \(+12 ext{ pts}\))
   - Sums base and penalties, clamped between 4 and 98.

6. **Intervention Priority Generation (`ai-service/app/services/stress_engine.py`)**: `get_recommendations()` calculates priority weights:

   - If screen \(> 5 ext{h}\), prioritize `"screen"` break (weight 4).
   - If sleep \(< 7 ext{h}\), prioritize `"sleep"` hygiene (weight 4).
   - If study \(> 7 ext{h}\), suppress study tips and boost relaxation. Returns sorted recommendations array.

7. **Persistence (`server/src/controllers/wellnessController.js`)**: The Node backend receives `{ score, level, recommendations }` from FastAPI and executes an upsert query in MongoDB:

```
await CheckIn.findOneAndUpdate(
  { userId: req.user._id, date: todayDate },
  checkInData,
  { new: true, upsert: true }
);
```

8. **Safety Watchdog & Alert Evaluation (`server/src/services/alertService.js`)**: `checkAndTriggerStressAlert(req.user, level)` inspects the student's recent check-in history. If consecutive high-stress entries exceed the student's auto-notify streak threshold, an emergency notification is dispatched via Nodemailer to the student's registered emergency contacts.

9. **Frontend Rendering (`client/src/App.jsx`)**: The React application receives the JSON response, updates state (`today.done = true`, `score = checkIn.stressScore`), renders the color-coded stress badge, displays personalized intervention action cards, and updates history charts.

PART 12 — STRESS LEVEL CLASSIFICATION

MindMate implements two distinct stress representation models across the codebase:

1. Runtime Multi-Tier Index (Check-In Endpoint & UI)

In the interactive daily application (`ai-service/app/services/stress_engine.py` and `client/src/App.jsx`), stress is quantified as an integer on a **100-point scale (4 to 98)**:

Score Range	Category Label	UI Color Accent	System Interpretation	Recommended Action
4 – 34	**Low**	Emerald Green (`#3F9E84`)	Balanced lifestyle and emotional stability	Maintain habits, protect regular sleep
35 – 64	**Moderate**	Indigo Blue (`#5B6FE8`)	Noticeable academic or physiological strain	Take targeted breaks, 2-min breathing exercises
65 – 98	**High**	Coral Red (`#D9737D`)	Elevated acute tension or cognitive overload	Step back from studies, contact support system

2. Clinical Classification Model (Trained ML Model)

In `ai-service/train_model.py` and `ai-service/data/student_stress_dataset.csv`, stress is categorized into 4 discrete ordinal classes based on psychological distress indicators:

- `0`: **No Stress** (Score baseline: 0–5)
- `1`: **Mild Stress** (Score baseline: 6–10)
- `2`: **Moderate Stress** (Score baseline: 11–15)
- `3`: **Severe Stress** (Score baseline: 16–21)

3. Physiological Classification (Wearable Model)

In `ai-service/app/services/wearable_engine.py`, continuous autonomic nervous system telemetry produces a 3-tier classification:

- `Low` (\(< 35.0\))
- `Medium` (\(35.0 - 64.9\))
- `High` (\(\ge 65.0\))

4. Critical Safety Thresholds

In `server/src/services/alertService.js`, the emergency notification watchdog operates on a dedicated safety threshold:

- A stress level of **High** occurring consecutively triggers the automated emergency welfare mechanism.

PART 13 — RECOMMENDATION SYSTEM

MindMate utilizes a **Multi-Modal Hybrid Recommendation Engine** combining heuristic behavioral prioritization, interactive somatic exercises, and generative conversational reasoning.

1. Deterministic Behavioral Prioritization Engine

- **File Location**: `ai-service/app/services/stress_engine.py` (`get_recommendations()`)
- **Strategy**: Dynamic weighting based on the student's lifestyle inputs.
- **Catalog of Interventions**:
    1. `breath` (2-minute paced breathing exercise — base weight 2)
    2. `walk` (10-minute fresh air outdoor walk — base weight 1)
    3. `screen` (Screen break / 20-20-20 rule — weight boosts to 4 when screen time \(> 5.0\) hours)
    4. `sleep` (Sleep hygiene protocol / screen curfew — weight boosts to 4 when sleep \(< 7.0\) hours)
    5. `focus` (Pomodoro 25-minute focused study session — suppressed to 0 if study time already \(> 7.0\) hours)
- **Execution**: Sorts the intervention catalog descending by weight, ensuring the student's immediate behavioral deficit (e.g., lack of sleep or screen fatigue) is shown at the very top.

2. Generative AI Empathetic Companion

- **File Location**: `ai-service/app/services/companion_engine.py`
- **Model**: Qwen 2.5 32B Instruct hosted via Groq Cloud API.
- **Mechanism**:
    - Injects student contextual state (e.g., "`Student has stress score 78 (High), slept 4.5 hours, exam in 2 days`").
    - Uses an empathetic counseling system prompt instructed to validate feelings, suggest micro-steps, and avoid clinical jargon.
    - Implements **Crisis Safeguard Detection**: If user inputs contain explicit self-harm or severe crisis keywords, the model overrides normal chatting and provides official crisis hotline contact numbers (988 / 741741).

3. Interactive Somatic Interventions (Client-Side)

- **BreathSession Modal** (`client/src/App.jsx`): Real-time visual lung expansion circle with 4-4-4-4 box breathing pacing (Inhale 4s, Hold 4s, Exhale 4s, Rest 4s) with countdown timer.
- **Audio Soundscapes** (`client/src/utils/`): Pure client-side Web Audio API synthesizer generating calibrated pink noise, rain, and oceanic frequencies without external media streams.

PART 14 — FRONTEND ARCHITECTURE

The frontend is an ultra-fast, responsive Single Page Application (SPA) built with **React 18**, **Vite**, and **Tailwind CSS**.

1. Component Hierarchy & Navigation

The core application interface is centralized within `client/src/App.jsx` and modular components under `client/src/components/`:

```
client/src/main.jsx
       |
       ▼
client/src/App.jsx
       |
       ├─► [Unauthenticated State]
       |     └─► AuthScreen (Sign In / Register toggle, form validation)
       |
       └─► [Authenticated State]
             ├─► Top Navigation Bar (Logo, current section title, notifications, profile)
             ├─► Main Content Container (Controlled by `tab` state):
             |     ├─► Dashboard (`tab === "dashboard"`)
             |     ├─► Wellness (`tab === "wellness"`)
             |     |     ├─► CheckIn (`wtab === "checkin"`)
             |     |     └─► Tips/Catalog (`wtab === "tips"`)
             |     ├─► AI Companion (`tab === "ai"`)
             |     ├─► History (`tab === "history"`)
             |     ├─► Emergency SOS (`tab === "sos"`)
             |     └─► Profile & Settings (`tab === "profile"`)
             ├─► Bottom Mobile Navigation Bar
             └─► Modals & Overlays:
                   ├─► BreathSession (Interactive animated breathing exercise)
                   └─► Toast Notification Banner
```

2. Primary Views Detailed:

- **Dashboard**: Displays greeting, stress score circular indicator, today's check-in status card, quick-action breathing exercise button, and high-stress warning banner if applicable.
- **Wellness & Check-In**:
  - 4 range sliders (Sleep 0-14h, Study 0-16h, Screen 0-16h, Exam 0-60d).
  - 5 emoji mood buttons with active visual indicators.
  - Free-form reflection textarea.
  - Interactive "Save & Calculate" button sending data to backend.
- **AI Companion**:
  - Full-screen conversational chat interface.
  - Bubble conversation history with user and assistant avatars.
  - Suggested prompt chips (e.g., *"Help me calm down"*, *"I can't focus on my exam"*, *"How to sleep better"*).
  - Typing indicator and auto-scroll to bottom.
- **History & Analytics**:
  - Interactive Recharts components (`AreaChart`, `LineChart`, `BarChart`).
  - Visualizes 7-day stress trajectory, mood distribution, and correlation between study hours and stress levels.
- **SOS & Emergency Contacts**:
  - Contact management cards (Name, Relationship, Phone, Email).
  - "Add Emergency Contact" modal with form validation.
  - High-Stress Auto-Notify toggle switch.
  - Immediate manual "Send SOS Alert" trigger button with confirmation modal.

3. State Management & API Communication

- **Token Storage**: JWT token persisted in browser `localStorage`.

- **API Service (`client/src/services/api.js`)**: Clean Axios wrapper managing authorization headers:

```
const token = localStorage.getItem("mm_token");
if (token) config.headers.Authorization = `Bearer ${token}`;
```

- **Offline Resiliency**: In the event of backend network latency, client-side fallback evaluation in `computeStress()` provides instant UI feedback without hanging.

PART 15 — BACKEND ARCHITECTURE

The primary backend is a **Node.js Express** server operating as an API Gateway and business logic coordinator located under `server/src/`.

1. Directory Structure & Responsibilities

- `server/src/server.js`: Process entry point. Binds HTTP listener to port 5000, establishes MongoDB connection via Mongoose, and handles graceful process termination.
- `server/src/app.js`: Express application bootstrap. Applies CORS, JSON body parser, registers route handlers, and sets up 404 and global error handling middleware.
- `server/src/routes/`: Route declarations defining HTTP verbs and endpoints:
  - `authRoutes.js`: `/api/auth/register`, `/api/auth/login`, `/api/auth/me`
  - `wellnessRoutes.js`: `/api/wellness/checkin`, `/api/wellness/today`, `/api/wellness/history`
  - `contactRoutes.js`: `/api/contacts` (GET, POST, DELETE)
  - `chatRoutes.js`: `/api/chat/message`
- `server/src/controllers/`: Business logic handlers:
  - `authController.js`: BCrypt password hashing, JWT generation, user lookup.
  - `wellnessController.js`: Daily check-in coordination, AI microservice HTTP forwarding, MongoDB upsert.
  - `contactController.js`: CRUD operations on emergency contact documents.
  - `chatController.js`: Proxies conversational messages to the Python AI service.
- `server/src/middleware/`:
  - `authMiddleware.js`: Intercepts incoming requests, validates `Bearer` JWT, attaches authenticated user to `req.user`.
  - `errorMiddleware.js`: Formats stack traces into consistent JSON error payloads.
- `server/src/services/`:
  - `aiService.js`: Axios HTTP client communicating with Python FastAPI service (`http://localhost:8000`).
  - `alertService.js`: High-stress streak evaluation and Nodemailer email dispatch.

2. Request Lifecycle Through Backend

```
HTTP Request: POST /api/wellness/checkin
      |
      ▼
app.js (Express Middleware: cors, json parser)
      |
      ▼
wellnessRoutes.js (protect middleware verifies JWT)
      |
      ▼
wellnessController.createOrUpdateCheckIn
      |
      ├─► aiService.computeStressWithAI() ──[HTTP POST]──► FastAPI :8000
      |                                                        |
      ├◄──────────────────[JSON Response]──────────────────────┘
      |
      ├─► CheckIn.findOneAndUpdate() ─────[Mongoose]──────► MongoDB :27017
      |
      ├─► alertService.checkAndTriggerStressAlert() ──► Nodemailer (if streak)
      |
      ▼
HTTP 200 Response: JSON { _id, userId, date, stressScore, recommendations... }
```

PART 16 — DATABASE ARCHITECTURE

The application uses **MongoDB** as its primary document database, connected through the **Mongoose ODM** (Object Data Modeling) library in Node.js.

1. Connection Lifecycle

- **Connection File**: `server/src/server.js`
- **URI Environment Variable**: `MONGODB_URI` (default: `mongodb://localhost:27017/mindmate`).
- **Connection Logic**:

```
mongoose.connect(process.env.MONGODB_URI || "mongodb://localhost:27017/mindmate", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});
```

- **Event Listeners**: Handles connection success (`mongoose.connection.on('connected')`) and database failover errors.

2. Database Collections & Schemas

A. Users Collection (`server/src/models/User.js`)

Stores student account credentials, profile settings, and welfare monitoring flags.

- **Fields**:
  - `name` (String, required, trimmed): Student's full name.
  - `email` (String, required, unique, lowercase, trimmed): Primary login identifier.
  - `password` (String, required, minlength: 8): BCrypt hashed password string.
  - `college` (String, default: `"Northfield University"`): Educational institution.
  - `preferences` (Object): User toggles for daily check-in reminders, wellness tips, and notifications.
  - `autoNotify` (Boolean, default: `true`): Permission toggle allowing system to alert emergency contacts.
  - `consecutiveHighStressDays` (Number, default: `0`): Counter tracking consecutive days with "High" stress.
  - `lastAlertSentAt` (Date, default: `null`): Timestamp of the last dispatched email alert (used for rate-limiting alerts).
  - `timestamps`: Automatic `createdAt` and `updatedAt`.
- **Hooks & Methods**:
  - `pre('save')`: Automatically salts and hashes the password before insertion if modified.
  - `matchPassword()`: Compares cleartext login candidate against the BCrypt hash.

B. CheckIns Collection (`server/src/models/CheckIn.js`)

Stores daily student lifestyle metrics, calculated stress scores, and recommendations.

- **Fields**:
  - `userId` (ObjectId, ref: `User`, required, indexed): Foreign reference to the student.
  - `date` (String, required): Calendar date string in ISO format (`YYYY-MM-DD`).
  - `sleep` (Number, required): Hours slept (0 to 24).
  - `study` (Number, required): Hours studied (0 to 24).
  - `screen` (Number, required): Screen entertainment time (0 to 24).
  - `exam` (Number, required): Days remaining until next exam.
  - `mood` (String, enum: `["great", "good", "okay", "stressed", "overwhelmed"]`, default: `"okay"`).
  - `note` (String, default: `""`): Qualitative reflection entry.
  - `stressScore` (Number, required): Calculated stress index (4 to 98).
  - `stressLevel` (String, enum: `["Low", "Moderate", "High"]`, required): Stress category.
  - `recommendations` (Array of Subdocuments): Stored list of suggested interventions (`id`, `title`, `blurb`, `mins`, `secs`, `tint`).
  - `timestamps`: `createdAt`, `updatedAt`.
- **Unique Compound Index**:

```
checkInSchema.index({ userId: 1, date: 1 }, { unique: true });
```

  Ensures each student has at most one record per calendar day. Subsequent updates on the same date perform an upsert (`findOneAndUpdate` with `upsert: true`).

C. Contacts Collection (`server/src/models/Contact.js`)

Stores the student's trusted safety network.

- **Fields**:
  - `userId` (ObjectId, ref: `User`, required, indexed): Student owner.
  - `name` (String, required, trimmed): Contact person's name.
  - `rel` (String, enum: `["Parent/Guardian", "Close Friend", "College Counselor"]`, required): Relationship type.
  - `email` (String, required, lowercase, trimmed): Destination email address for automated emergency alerts.
  - `phone` (String, required, trimmed): Emergency phone contact.
  - `timestamps`: `createdAt`, `updatedAt`.

PART 17 — PRIVACY & SECURITY

Student wellness applications handle sensitive personal data requiring strict security measures.

1. Existing Security Mechanisms in Codebase

- **Password Hashing**: User passwords are encrypted using `bcryptjs` with 10 salt rounds before storage in MongoDB. Raw passwords are never stored in plaintext or logged.
- **Stateless Token Authentication**: Authentication is maintained via JSON Web Tokens (JWT) signed with a server-side secret (`JWT_SECRET`). Tokens expire after a configured duration (e.g., 30 days) and must be presented in the HTTP `Authorization: Bearer <token>` header.
- **CORS Access Control**: Both the Express API gateway (`cors()`) and the FastAPI AI service (`CORSMiddleware`) restrict origin headers, preventing malicious cross-site scripting from unauthorized domains.
- **Model Key Sequestration**: The Groq Cloud API credentials (`GROQ_API_KEY`) and SMTP credentials reside strictly in backend environment variables (`.env`) and are never exposed to the frontend browser bundle.
- **Opt-in Safety Notifications**: The automated emergency contact notification feature is guarded by student consent (`autoNotify: true` toggle in Profile/SOS) and rate-limited by a 24-hour cooldown (`lastAlertSentAt`).

2. Areas Required for Production Hardening

- **Data Encryption at Rest**: In production, MongoDB Atlas or enterprise databases must enable AES-256 encryption at rest for all collections containing student notes and mental health logs.
- **Rate Limiting**: Add `express-rate-limit` middleware to `/api/auth/login` and `/api/wellness/checkin` to prevent brute-force credential stuffing and API abuse.
- **HIPAA / FERPA / GDPR Compliance**: If deployed within an educational institution, data governance must comply with FERPA (student educational records) and regional health privacy laws, providing data export and permanent account deletion endpoints.

PART 18 — GENERAL SYSTEM LIMITATIONS

To maintain engineering and academic integrity, the following architectural and operational limitations must be recognized:

1. **Self-Report Subjectivity**: The system relies on student inputs (sleep hours, mood selection, journal notes). Self-reported inputs are subject to recall bias, situational exaggeration, or under-reporting due to stigma.
2. **Calendar-Day Granularity**: Check-ins are keyed by date (`YYYY-MM-DD`). The system does not capture rapid diurnal fluctuations, midday panic attacks, or circadian rhythm anomalies unless logged manually.
3. **No Clinical Diagnostic Ability**: The system cannot diagnose clinical disorders (such as Major Depressive Disorder, Generalized Anxiety Disorder, Bipolar Disorder, or PTSD). It is strictly a lifestyle tracking and support tool.
4. **LLM Generation Stochasticity**: The AI Companion utilizes Qwen 2.5 on Groq. While steered by strict system prompts and crisis detection rules, generative models can occasionally generate repetitive advice or misinterpret idiomatic expressions.
5. **Hardware Telemetry Simulation**: The wearable classification logic (`wearable_engine.py`) exists in code, but real-time BLE (Bluetooth Low Energy) streaming from smartwatches is not currently integrated into the frontend UI.

6. **Email Relay Dependency**: Emergency welfare alerts require a working SMTP relay (`nodemailer` with Gmail, SendGrid, or AWS SES). If SMTP credentials fail, alerts will log errors without guaranteed delivery.

PART 19 — MACHINE LEARNING LIMITATIONS SPECIFICALLY

The machine learning implementation in `ai-service/train_model.py` and `ai-service/models/` has specific technical boundaries:

1. Offline Dataset vs. Online Check-In Decoupling

- **The Reality**: The trained Random Forest Classifier expects 7 clinical GAD-7 psychological survey questions (`Nervousness`, `Unable to control`, `Worry`, `Trouble in Relaxation`, `Restlessness`, `Irritability`, `Fear`).
- The daily interactive UI, however, collects lifestyle habits (`sleep`, `study`, `screen`, `exam`, `mood`, `note`) and evaluates them via `stress_engine.compute_stress_score()`.
- **Consequence**: The offline ML model evaluates standardized psychometric screening surveys, while the daily check-in evaluates day-to-day behavioral lifestyle habits.

2. Severe Class Imbalance in Training Data

- While the overall dataset contains 16,384 samples, the classes are distributed as:
  - Mild: 48.0%
  - Moderate: 45.5%
  - Severe: 4.5% (737 samples)
  - No Stress: 2.0% (325 samples)
- Although mitigated with `class_weight="balanced"`, the model had fewer examples to learn nuanced decision boundaries for acute, severe crisis states.

3. Static Tabular Architecture (Lack of Temporal Sequence Modeling)

- The Random Forest treats each student survey record as an independent point in feature space.
- It does not incorporate sequence memory (such as Recurrent Neural Networks, LSTMs, or Transformers) to track whether a student has been deteriorating over 3 consecutive weeks.

4. Absence of Biological Ground Truth

- Labels in the training dataset are based on subjective self-report questionnaires, not physiological biomarkers (cortisol levels, continuous HRV measurements, or clinical diagnostic interviews).

PART 20 — "WHAT IS ACTUALLY AI HERE?" (TRANSPARENT TECHNICAL BREAKDOWN)

In technical interviews and viva presentations, credibility depends on being 100% transparent about which components use AI/ML and which use standard software engineering logic.

Component	Technology	Is It Actually AI / ML?	Detailed Explanation
Student Stress Classifier (`train_model.py`)	Scikit-Learn `RandomForestClassifier`	**YES (Supervised ML)**	Ensemble of 120 decision trees trained on 16,384 samples to classify 4 stress levels with 85.2% accuracy.
Continuous Stress Regressor (`train_model.py`)	Scikit-Learn `RandomForestRegressor`	**YES (Supervised ML)**	Ensemble of 100 decision trees trained to output continuous stress severity numbers.
Conversational Companion (`companion_engine.py`)	Qwen 2.5 32B via Groq Cloud API	**YES (Generative AI / LLM)**	Deep neural network language model generating empathetic multi-turn conversation and performing crisis intent triage.
Physiological Classifier (`wearable_engine.py`)	Serialized Pickle Model (`.pkl`)	**YES (Pretrained ML)**	Supervised model accepting Heart Rate, Galvanic Skin Response, and Skin Temperature telemetry.
Daily Stress Score (`stress_engine.py`)	Deterministic Arithmetic Formula	**NO (Rule-Based Heuristic)**	Mathematical penalty formula factoring sleep deficit, study fatigue, screen time, exam proximity, and keyword matches.
Recommendation Selector (`stress_engine.py`)	Priority Weight Sorting	**NO (Rule-Based Algorithm)**	Conditional Python dictionary assigning weights (e.g., `screen > 5h -> weight 4`) and sorting the intervention array.
Emergency SOS Watchdog (`alertService.js`)	MongoDB query + Nodemailer	**NO (Backend Business Logic)**	Database query checking if `consecutiveHighStressDays >= 3` and triggering automated SMTP emails.
Breathing Circle & Soundscapes (`App.jsx`)	CSS Keyframes & Web Audio API	**NO (Frontend Multimedia)**	Browser-native canvas, animation timers, and audio oscillators for relaxation.

> *[!TIP]* ***Viva Tip****: When professors ask "Where is the AI in your project?", answer: "Our project uses a multi-tier AI architecture: First, an offline supervised Random Forest Classifier trained on 16,384 GAD-7 clinical survey records achieving 85.2% accuracy. Second, a cloud-hosted Generative AI model (Qwen 2.5 on Groq) for contextual, empathetic mental health conversation and crisis triage. Third, a pre-trained physiological model for wearable telemetry. For instantaneous, deterministic daily habit check-ins, we complement these models with a transparent multi-factor heuristic engine."*

PART 21 — HOW TO RUN THE PROJECT LOCALLY

To run the complete MindMate ecosystem on your machine, you need three concurrent processes: the MongoDB database, the Python FastAPI AI service, the Node.js Express API gateway, and the Vite React frontend.

Step 1: Ensure Prerequisites

- **Node.js**: v18.0.0 or higher (`node -v`)
- **Python**: v3.10, 3.11, or 3.12 (`python3 --version`)
- **MongoDB**: Community Server installed and active on local port 27017 (`mongod` service active)

Step 2: Start the Python AI Microservice (Port 8000)

Open Terminal 1:

```
cd /mnt/E_Drive/Projects/MindMate/ai-service

# Create and activate virtual environment (if not already created)
python3 -m venv .venv
source .venv/bin/activate

# Install required dependencies
pip install -r requirements.txt

# (Optional) Retrain and export ML models from the dataset
python train_model.py

# Launch FastAPI ASGI server
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

- Verify health: Open `http://localhost:8000/health` in your browser. It should return `{"status":"healthy"}`.

Step 3: Start the Node.js Express API Gateway (Port 5000)

Open Terminal 2:

```
cd /mnt/E_Drive/Projects/MindMate/server

# Install backend dependencies
npm install

# Start the Express server
node src/server.js
# Or for live auto-reload during development:
npm run dev
```

- Verify health: Open `http://localhost:5000/health`. It should return `{"status":"API Gateway active"}`.

Step 4: Start the React Frontend Application (Port 5173)

Open Terminal 3:

```
cd /mnt/E_Drive/Projects/MindMate/client

# Install frontend dependencies
npm install

# Start Vite development server
npm run dev -- --host 0.0.0.0 --port 5173
```

- Access the web application: Open `http://localhost:5173` in any modern web browser.

PART 22 — DEBUGGING & COMMON ISSUES

When setting up or demonstrating the application, you may encounter the following common issues:

1. `Error: connect ECONNREFUSED 127.0.0.1:8000`

- **Root Cause**: The Node.js server cannot communicate with the Python AI microservice.
- **Fix**: Ensure Terminal 1 is running `uvicorn app.main:app --port 8000`. Test endpoint manually: `curl http://localhost:8000/health`.

2. `MongooseServerSelectionError: connect ECONNREFUSED 127.0.0.1:27017`

- **Root Cause**: The local MongoDB daemon is not running.
- **Fix**: Start the MongoDB service (`sudo systemctl start mongod` on Linux or check Windows Services).

3. Blank White Screen in Browser

- **Root Cause**: Unhandled client-side JavaScript runtime exception or missing environment configuration.
- **Fix**: Open Browser Developer Tools (`F12`), check the **Console** tab for syntax errors or failed imports. Clear `localStorage.removeItem("mm_token")` if a malformed session token is cached.

4. Groq Companion Chat Returns Fallback Message

- **Root Cause**: `GROQ_API_KEY` is missing or invalid in `ai-service/.env`.
- **Fix**: Verify your `.env` contains a valid key: `GROQ_API_KEY=gsk_...`. The system is built with defensive resilience: if the API call fails or times out, `companion_engine.py` gracefully returns a supportive local fallback reply rather than crashing.

5. CORS Blocked on `/wellness/compute-stress` or `/api/*`

- **Root Cause**: Frontend running on a port not listed in `CORSMiddleware` or Express `cors()`.
- **Fix**: Check `ai-service/app/core/config.py` where allowed origins are defined (`http://localhost:5173`, `http://localhost:3000`).

PART 23 — TOP 20 MOST IMPORTANT FILES IN THE PROJECT

#	File Path	Subsystem	Why It Is Critical / What Breaks If Removed

#	File Path	Subsystem	Why It Is Critical / What Breaks If Removed
1	`client/src/App.jsx`	Frontend	Core Single Page Application containing all navigation, state, check-in forms, charts, and modals. Removing it destroys the entire UI.
2	`server/src/server.js`	Backend	Entry point initializing Node HTTP listener and connecting to MongoDB. The backend cannot start without it.
3	`server/src/app.js`	Backend	Express application configurator defining middleware, CORS, and route mounting.
4	`ai-service/app/main.py`	AI Service	FastAPI application entry point setting up ASGI lifecycle, CORS, and endpoint routers.
5	`ai-service/train_model.py`	ML Pipeline	Supervised training script that ingests the 16,384 dataset, scales features, fits Random Forest models, and exports joblib artifacts.
6	`ai-service/data/student_stress_dataset.csv`	ML Data	Ground truth dataset (16,384 records) containing GAD-7 clinical psychological stress variables.
7	`ai-service/app/services/stress_engine.py`	AI Engine	Implements deterministic stress calculation (`compute_stress_score`) and prioritized recommendations (`get_recommendations`).
8	`ai-service/app/services/companion_engine.py`	GenAI	Orchestrates Qwen 2.5 on Groq Cloud, handles prompt injection, and executes crisis intent safety triage.
9	`server/src/controllers/wellnessController.js`	Backend	Orchestrates daily check-ins, communicates with FastAPI, saves records in MongoDB, and triggers alerts.
10	`server/src/models/CheckIn.js`	Database	Mongoose schema defining daily check-in attributes, stress levels, compound unique indexes, and recommendations.
11	`server/src/models/User.js`	Database	Mongoose schema for student accounts, BCrypt password hashing hooks, and emergency streak counters.
12	`server/src/models/Contact.js`	Database	Mongoose schema for emergency contacts (parents, counselors, friends).
13	`server/src/services/alertService.js`	Backend	Watchdog monitoring continuous high-stress streaks and sending automated emergency emails via Nodemailer.
14	`server/src/services/aiService.js`	Backend	Axios client proxying wellness and chat requests from Node gateway to Python FastAPI.
15	`server/src/middleware/authMiddleware.js`	Backend	JWT authorization filter protecting secure student endpoints.
16	`client/src/services/api.js`	Frontend	Axios client attaching JWT Bearer tokens to all outbound browser API requests.
17	`ai-service/app/routes/wellness.py`	AI Service	FastAPI endpoint exposing `/wellness/compute-stress` to the backend gateway.
18	`ai-service/app/routes/chat.py`	AI Service	FastAPI endpoint exposing `/chat/message` for AI Companion chat.

#	File Path	Subsystem	Why It Is Critical / What Breaks If Removed
19	`ai-service/models/student_stress_rfc.joblib`	ML Model	Serialized 120-tree Random Forest Classifier binary.
20	`server/src/controllers/authController.js`	Backend	Handles student registration, BCrypt password comparison, and JWT creation.

PART 24 — 42 PROJECT VIVA QUESTIONS & ANSWERS

This comprehensive question-and-answer repository prepares you for technical interviews, university viva examinations, and project evaluations.

Section A: Project Overview & Objectives (Q1 – Q6)

Q1: What is the main objective of the MindMate project?

Answer: MindMate is an AI-driven student mental wellness platform designed to proactively detect academic burnout and emotional distress, provide tailored lifestyle and breathing interventions, and offer an empathetic AI conversational companion.

Q2: What exact problem in higher education does this project address?

Answer: College students frequently face academic overload, sleep deprivation, and exam pressure. Stigma and scarcity of counseling staff cause students to seek help only after reaching severe crisis. MindMate provides accessible, non-stigmatizing daily screening and early intervention.

Q3: Is MindMate a clinical diagnostic tool?

Answer: No. MindMate is strictly an educational, lifestyle, and screening support system. It is not FDA/CE-cleared and does not diagnose psychiatric conditions such as Major Depressive Disorder or Generalized Anxiety Disorder.

Q4: Who is the intended target user?

Answer: Undergraduate, graduate, and high school students experiencing academic stress, irregular sleep, or exam anxiety.

Q5: What makes this project "AI-driven" rather than a simple web questionnaire?

Answer: It integrates multiple AI layers: an ensemble Random Forest Classifier trained on 16,384 psychological screening samples, a continuous Random Forest Regressor, an autonomic wearable signal model, and a cloud-hosted Large Language Model (Qwen 2.5 32B) for conversational context and crisis triage.

Q6: How does the system handle students who are in acute psychological crisis?

Answer: Through a multi-tiered safety protocol: the AI companion has keyword-based crisis detection that immediately displays national hotlines (988), and the backend watchdog notifies designated emergency contacts if high stress persists for 3 consecutive days.

Section B: Architecture & Microservices (Q7 – Q12)

Q7: Why did you split the backend into Node.js Express and Python FastAPI instead of using a single server?

Answer: Separation of concerns. Node.js with Express excels at asynchronous I/O, user authentication, session security, and database CRUD. Python is the industry standard for scientific computing, Scikit-Learn model execution, and LLM orchestration. A microservice architecture decouples business logic from computational AI inference.

Q8: How do the Node.js gateway and Python AI service communicate?

Answer: Through internal RESTful HTTP calls over a private network interface (Node makes HTTP POST requests via Axios to `http://localhost:8000`).

Q9: What happens if the Python AI service goes down? Does the whole application crash?

Answer: No. The architecture is defensively isolated. If the AI service fails or times out, the Node backend and React frontend fall back to deterministic local heuristic calculations, ensuring the student's daily check-in is still safely recorded.

Q10: What is the role of the frontend Single Page Application (SPA)?

Answer: The React SPA manages the presentation layer, form interactions, animated breathing exercises, real-time Recharts visualizations, and client-side token management, entirely decoupled from data persistence.

Q11: What is the role of MongoDB in this architecture?

Answer: MongoDB provides high-throughput document persistence for student profiles, flexible check-in histories, and emergency contact lists with compound indexing.

Q12: How are static assets and soundscapes served without overloading the server?

Answer: Soundscapes are synthesized directly on the client machine using the browser's native Web Audio API oscillators, requiring zero bandwidth and zero external media server streaming.

Section C: Machine Learning & Dataset (Q13 – Q20)

Q13: What machine learning algorithm is used in `train_model.py` and why?

Answer: `RandomForestClassifier`. Random Forest is an ensemble of decision trees utilizing bootstrap aggregation (bagging). It handles non-linear interactions between psychological factors exceptionally well, is robust against outliers, and avoids overfitting via feature subsampling.

Q14: What dataset did you use to train the model?

Answer: A clinical student psychological stress dataset containing 16,384 records and 8 attributes based on standardized GAD-7 anxiety and stress inventory metrics.

Q15: What are the input features of the machine learning model?

Answer: Exactly 7 features: `Nervousness`, `Unable to control`, `Worry`, `Trouble in Relaxation`, `Restlessness`, `Irritability`, and `Fear`.

Q16: What is the target variable?

Answer: `Stress`, categorized into four ordinal classes: `No Stress` (0), `Mild Stress` (1), `Moderate Stress` (2), and `Severe Stress` (3).

Q17: How did you handle categorical variables in the dataset?

Answer: Since the survey answers follow an ordered progression (`No` < `Mild` < `Moderate` < `Severe`), we applied ordinal integer mapping: `{"No": 0, "Mild": 1, "Moderate": 2, "Severe": 3}`.

Q18: Did the dataset have missing values, and how were they treated?

Answer: The 16,384-record dataset was clean with zero null values across all columns.

Q19: Why was feature scaling applied using `StandardScaler`?

Answer: `StandardScaler` standardizes features to zero mean and unit variance (\(z = (x - \mu)/\sigma\)). While tree splits are invariant to monotonic scaling, scaling is required for regularized regressors and ensures numerical stability across pipelines.

Q20: How was the dataset split between training and testing?

Answer: An 80/20 train/test split (13,107 train samples, 3,277 test samples) using `train_test_split` with `random_state=42` and `stratify=y` to preserve the exact class distribution in both splits.

Section D: Preprocessing & Evaluation Metrics (Q21 – Q26)

Q21: What accuracy did your Random Forest model achieve?

Answer: The classifier achieved **85.20% accuracy** on the holdout test set of 3,277 unseen student samples.

Q22: What were the precision, recall, and F1-score?

Answer: Weighted Precision was **86.74%**, Weighted Recall was **85.20%**, and Weighted F1-Score was **85.72%**.

Q23: Why is weighted F1-score more important than simple accuracy here?

Answer: The dataset has significant class imbalance (4.5% Severe, 2.0% No Stress vs 93.5% Mild/Moderate). Simple accuracy can be deceptively high by predicting majority classes; the weighted F1-score accounts for false positives and false negatives across all classes.

Q24: How did you address the class imbalance during training?

Answer: By passing `class_weight="balanced"` to the `RandomForestClassifier`. This mathematically penalizes misclassifications of minority classes (like Severe Stress) inversely proportional to their representation.

Q25: Which feature had the highest importance according to the model?

Answer: `Fear` had the highest Gini feature importance at 15.43%, closely followed by `Restlessness` at 14.59% and `Nervousness` at 14.37%.

Q26: How are the trained models saved and loaded?

Answer: Using `joblib.dump()` to serialize the fitted estimator into `student_stress_rfc.joblib` and `joblib.load()` during inference.

Section E: Generative AI & AI Companion (Q27 – Q32)

Q27: Which Large Language Model powers the AI Companion?

Answer: `Qwen 2.5 32B Instruct`, hosted on Groq Cloud's LPUs (Language Processing Units) for low-latency responses.

Q28: How is student context provided to the LLM?

Answer: Through dynamic system prompt context injection. The backend formats recent stress score, sleep hours, exam proximity, and mood into the system prompt before user messages are evaluated.

Q29: What guardrails prevent the AI Companion from giving medical diagnoses?

Answer: System prompt instructions explicitly prohibit prescribing medication, offering clinical diagnoses, or claiming medical authority, directing the model to act as a supportive peer counselor.

Q30: What safety mechanism exists for self-harm or suicide indicators?

Answer: Conversational triage logic in `companion_engine.py` scans inputs for crisis keywords. When triggered, the assistant prioritizes crisis intervention, providing immediate contact info for the 988 Suicide & Crisis Lifeline.

Q31: Why did you choose Groq Cloud for the LLM?

Answer: Groq's LPU architecture delivers response speeds of over 300 tokens per second, ensuring students experiencing anxiety receive near-instantaneous responses.

Q32: What happens if Groq API rate limits or network issues occur?

Answer: A `try/except` block catches the exception and returns a warm, pre-configured empathetic fallback response so the chat interface never errors out.

Section F: Database, Security & SOS Watchdog (Q33 – Q37)

Q33: How does the application ensure a student only has one check-in per day?

Answer: Via a compound unique index in MongoDB: `checkInSchema.index({ userId: 1, date: 1 }, { unique: true })`. At the application level, `CheckIn.findOneAndUpdate(..., { upsert: true })` updates today's existing record if submitted again.

Q34: How are passwords secured in the database?

Answer: Passwords are encrypted using `bcryptjs` with 10 salt rounds in a Mongoose `pre('save')` hook. The database only ever stores non-reversible one-way cryptographic hashes.

Q35: How does stateless JWT authentication work across requests?

Answer: Upon login, the server signs a JWT containing the user ID with `JWT_SECRET`. The client stores it in `localStorage` and includes it in the `Authorization: Bearer <token>` header for all authenticated requests, verified by `authMiddleware.js`.

Q36: How does the Emergency Alert Watchdog work?

Answer: In `server/src/services/alertService.js`, every check-in evaluates `consecutiveHighStressDays`. If the student reaches 3 consecutive high-stress days and has `autoNotify: true` enabled, Nodemailer automatically dispatches an alert email to the student's designated emergency contacts.

Q37: How do you prevent emergency contacts from being spammed with emails?

Answer: Rate-limiting via `lastAlertSentAt`. The system enforces a 24-hour cooldown period between consecutive automated email dispatches.

Section G: Deployment, Ethical Realities & Future Scope (Q38 – Q42)

Q38: How can this full-stack project be deployed to production?

Answer:

- Frontend: Static build (`npm run build`) hosted on Vercel or Netlify.
- Node.js Gateway: Containerized or hosted on Render/Railway.
- Python AI Service: Docker container on Render, Railway, or AWS ECS.
- Database: Managed MongoDB Atlas cluster with TLS encryption.

Q39: What is the difference between your offline ML model and the online check-in calculation?

Answer: The offline Random Forest model evaluates standardized GAD-7 clinical survey responses (7 items, 16,384 samples). The daily check-in evaluates everyday lifestyle habits (sleep hours, study hours, screen time, exam countdown, journal notes) via a transparent multi-factor heuristic formula.

Q40: What ethical considerations surround AI mental health applications?

Answer: Data privacy, student consent, prevention of over-reliance on AI companions, and ensuring users understand AI cannot replace human medical professionals or licensed therapists.

Q41: If you had 3 more months, what would you improve?

Answer:

1. Implement real-time Bluetooth Low Energy (BLE) streaming from consumer smartwatches for continuous heart-rate variability (HRV) stress tracking.
2. Replace static Random Forest with a recurrent temporal model (LSTM or Transformer) to analyze stress trajectories over time.
3. Integrate federated learning to allow on-device model personalization without centralizing sensitive student health notes.

Q42: Summarize your project in two sentences for an interviewer.

Answer: MindMate is a full-stack mental wellness platform combining a Random Forest classifier trained on 16,384 student screening samples with an LLM-powered empathetic companion and a proactive emergency watchdog. It translates daily student habits and psychometrics into actionable lifestyle interventions and safety alerts before academic stress escalates into clinical burnout.

PART 25 — BEGINNER-FRIENDLY EXPLANATION (EXPLAIN IT LIKE I'M 12)

If you have never written a line of code or studied machine learning, here is how you can understand and explain MindMate to anyone:

The Problem:

Imagine being a college student during final exams week. You slept only 4 hours, spent 10 hours staring at a computer screen, and your hardest exam is in two days. You feel overwhelmed, exhausted, and anxious, but you don't know who to talk to, and university counseling appointments take weeks to schedule.

What MindMate Does:

Think of MindMate as a **smart digital school companion** that lives on your phone or laptop. It does three simple things every day:

1. **The Morning Check-In (Like a friendly health nurse)**: You move a few simple sliders telling it how many hours you slept, how much you studied, how much screen time you had, and how many days until your next test. You pick a quick emoji for your mood.
2. **The Smart Assessment (The "Brain")**: Behind the scenes, the computer analyzes your answers. It calculates a "Stress Score" from 0 to 100 and tells you whether you are in the **Low**, **Moderate**, or **High** stress zone.
    - If you slept very little, it knows your body is physically tired.
    - If exams are tomorrow, it knows your mind is under pressure.
    - It also uses an AI model trained on over 16,000 real student survey records to recognize patterns of nervousness and anxiety.
3. **Actionable Micro-Steps (Not just a score)**: Instead of just giving you a number, MindMate gives you immediate, easy micro-steps to feel better right now:
    - An animated 2-minute breathing exercise circle that teaches your body to calm down.
    - Suggestions like taking a 10-minute walk without your phone or stopping screen time 30 minutes before bed.
4. **The Empathetic AI Friend**: If you want to talk about how stressed you are, you can open the chat. A kind, non-judgmental AI companion listens to your worries and helps you break down big study tasks into small, manageable steps.
5. **The Safety Net (The Guardian Angel)**: If you log into the app and have severe high stress for 3 days in a row, the app quietly reaches out to your designated safety contact (like your mom, roommate, or campus counselor) with an email saying: *"Hey, your friend is having a tough few days. Please give them a call to check in on them."* This ensures no student suffers alone in silence.

PART 26 — 7-DAY STEP-BY-STEP LEARNING ROADMAP

Follow this 7-day plan to achieve 100% mastery over the MindMate codebase before presenting it:

```
┌──────────────────────────────────────────────────────────────────┐
│                  MINDMATE 7-DAY MASTERY ROADMAP                  │
├──────────┬───────────────────────────────────────────────────────┤
│ Day 1    │ Run the System & Explore the User Journey             │
│ Day 2    │ Master the Machine Learning Pipeline (train_model.py) │
│ Day 3    │ Deep Dive into Python FastAPI & AI Engines            │
│ Day 4    │ Understand the Node.js Express Gateway & MongoDB Schemas │
│ Day 5    │ Master React Frontend State & UI Components (App.jsx) │
│ Day 6    │ Understand Safety Mechanisms (Watchdog, Groq LLM, Nodemailer)│
│ Day 7    │ Rehearse the 42 Viva Questions & Practice Live Demo   │
└──────────┴───────────────────────────────────────────────────────┘
```

Day 1: Run the System & Explore the User Journey

- Open three terminal tabs.
- Start Python AI Service (`uvicorn app.main:app --port 8000`).
- Start Node.js API Gateway (`node src/server.js` on port 5000).
- Start Vite React frontend (`npm run dev` on port 5173).
- Open browser, create a new student account, fill out the daily check-in, open the AI companion, try the breathing circle, and add an emergency contact.

Day 2: Master the Machine Learning Pipeline

- Open `ai-service/data/student_stress_dataset.csv`. Inspect the 8 columns and 16,384 rows.
- Read `ai-service/train_model.py` line-by-line.
- Understand `StandardScaler`, `train_test_split(stratify=y)`, and `RandomForestClassifier(class_weight='balanced')`.
- Run `python train_model.py` and inspect `model_metrics.json`.

Day 3: Deep Dive into Python FastAPI & AI Engines

- Read `ai-service/app/main.py` and observe the CORS middleware.
- Read `ai-service/app/routes/wellness.py` and `ai-service/app/services/stress_engine.py`.
- Trace the math in `compute_stress_score()`: calculate penalties for sleep, study, screen, exam, and note keywords.
- Read `ai-service/app/services/companion_engine.py`: see how the Groq API key is used to call Qwen 2.5 with conversational crisis triage.

Day 4: Understand Node.js Express Gateway & MongoDB Schemas

- Open `server/src/server.js` and `server/src/app.js`.
- Read `server/src/models/User.js`, `CheckIn.js`, and `Contact.js`. Understand why the compound unique index `{ userId: 1, date: 1 }` prevents duplicate daily check-ins.
- Read `server/src/controllers/wellnessController.js` and trace how `computeStressWithAI()` communicates with FastAPI.

Day 5: Master React Frontend State & UI Components

- Open `client/src/App.jsx`.
- Trace the `tab` state (`dashboard`, `wellness`, `ai`, `history`, `sos`, `profile`).
- See how `client/src/services/api.js` attaches the JWT token from `localStorage`.
- Trace how the Recharts charts render historical data and how the `BreathSession` modal animates box breathing.

Day 6: Safety Watchdog, Crisis Safeguards, & Production Hardening

- Read `server/src/services/alertService.js`: observe how `consecutiveHighStressDays >= 3` triggers automated email alerts via Nodemailer.
- Verify the 24-hour rate limit (`lastAlertSentAt`) and the crisis keyword interception in `companion_engine.py`.
- Review Part 17 (Privacy & Security) and Part 18 (Limitations).

Day 7: Rehearse Viva Questions & Deliver the Presentation

- Read Part 24 (42 Project Viva Questions & Answers) aloud.
- Practice answering the 3 most common professor questions:
    1. *"Where is the actual AI in your project?"* (Answer using Part 20 table).
    2. *"How does data flow from the slider to the database?"* (Answer using Part 11).
    3. *"Why use two backends instead of one?"* (Answer using Q7 in Part 24).
- Rehearse the 2-minute elevator pitch from Q42.

PART 27 — FINAL CHEAT SHEET / QUICK REFERENCE

Keep this reference card open during presentations, technical interviews, or viva exams:

1. Network & Port Map

Component	Port	Technology	Process Command	Health URL
Frontend UI	`5173`	React 18 / Vite	`npm run dev -- --port 5173`	`http://localhost:5173`
API Gateway	`5000`	Node.js / Express	`node src/server.js`	`http://localhost:5000/health`
AI Microservice	`8000`	Python / FastAPI	`uvicorn app.main:app --port 8000`	`http://localhost:8000/health`
Database	`27017`	MongoDB	`mongod`	`mongodb://localhost:27017`

2. Key Thresholds & Business Constants

Parameter	Value	Location in Code	Purpose
Low Stress	`Score < 35`	`stress_engine.py`, `App.jsx`	Green zone, healthy baseline
Moderate Stress	`35 <= Score < 65`	`stress_engine.py`, `App.jsx`	Blue zone, lifestyle fatigue
High Stress	`Score >= 65`	`stress_engine.py`, `App.jsx`	Red zone, acute distress
Emergency Streak	`3 consecutive days`	`server/src/services/alertService.js`	Triggers automated email alert to contacts
Alert Cooldown	`24 hours`	`server/src/services/alertService.js`	Prevents email spamming to emergency contacts
Box Breathing	`4s Inhale, 4s Hold, 4s Exhale, 4s Hold`	`client/src/App.jsx`	Somatic relaxation exercise
Crisis Hotlines	`988 (Call/Text), 741741 (Text HOME)`	`companion_engine.py`, `App.jsx`	Real-world emergency crisis resources

3. Machine Learning Specifications

Property	Value
Algorithm	`RandomForestClassifier` (120 trees, max depth 10, balanced class weight)
Dataset Size	16,384 rows × 8 columns (`ai-service/data/student_stress_dataset.csv`)
Validation Scheme	Stratified 80/20 train/test split (13,107 train / 3,277 test)
Accuracy	**85.20%**
Weighted F1-Score	**85.72%**
Top Feature	`Fear` (15.43% importance), `Restlessness` (14.59%), `Nervousness` (14.37%)
LLM Model	Qwen 2.5 32B Instruct on Groq Cloud LPUs

4. Golden Rules for Interview Success

1. **Never claim clinical diagnosis**: Always describe the platform as a student wellness screening and lifestyle intervention tool.

2. **Be completely honest about AI vs. Rules**: Emphasize that the Random Forest model evaluates psychological surveys, Groq Qwen powers empathetic chat, and daily check-ins use a transparent multi-factor heuristic formula.
3. **Highlight the safety net**: Mention that the high-stress watchdog and crisis triage elevate the project from a simple dashboard to a responsible, real-world application.